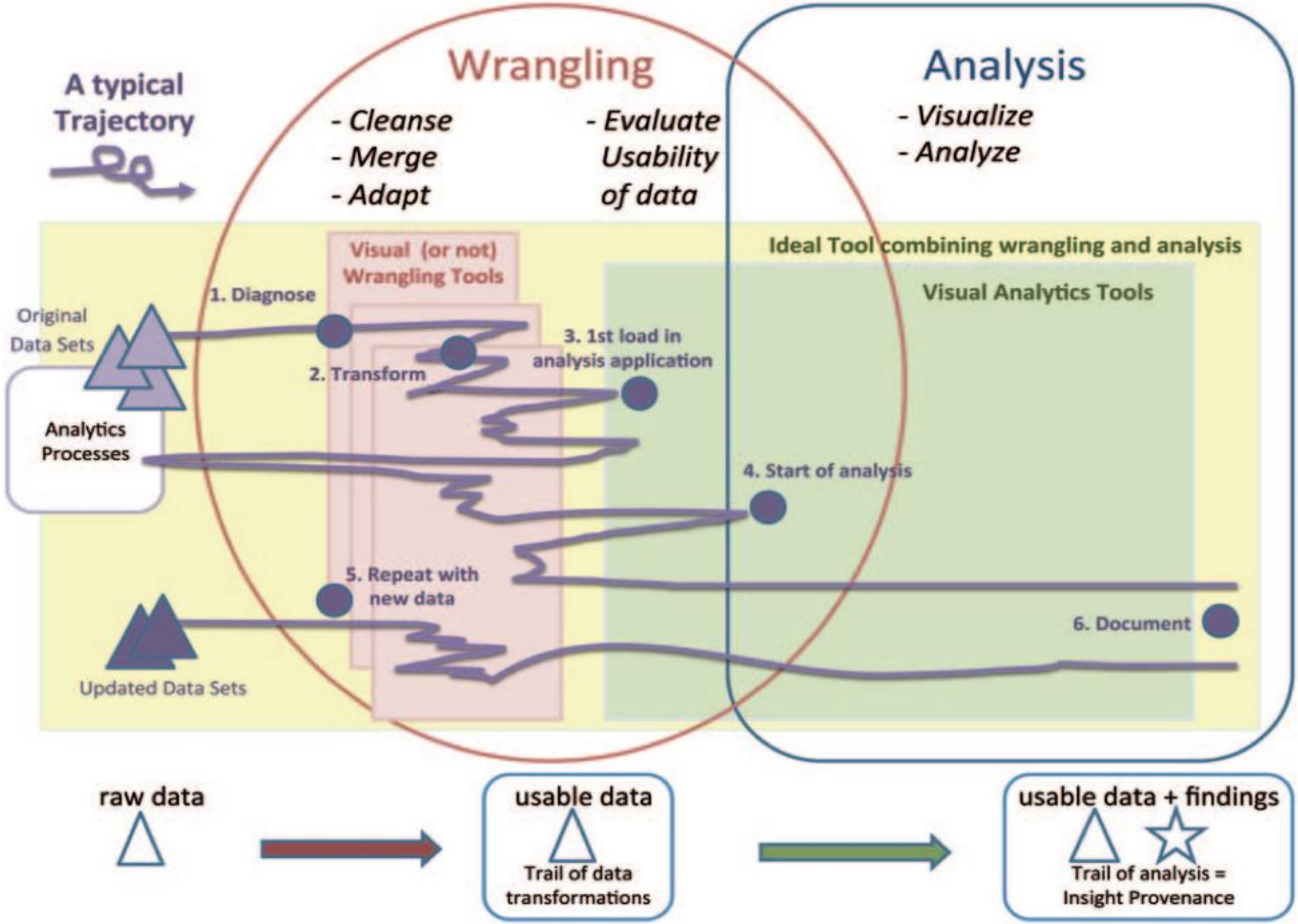
DSCI-560
Lecture 8: Data Extraction
Data Science Professional Practicum

Young Cho
Department of Electrical Engineering
University of Southern California

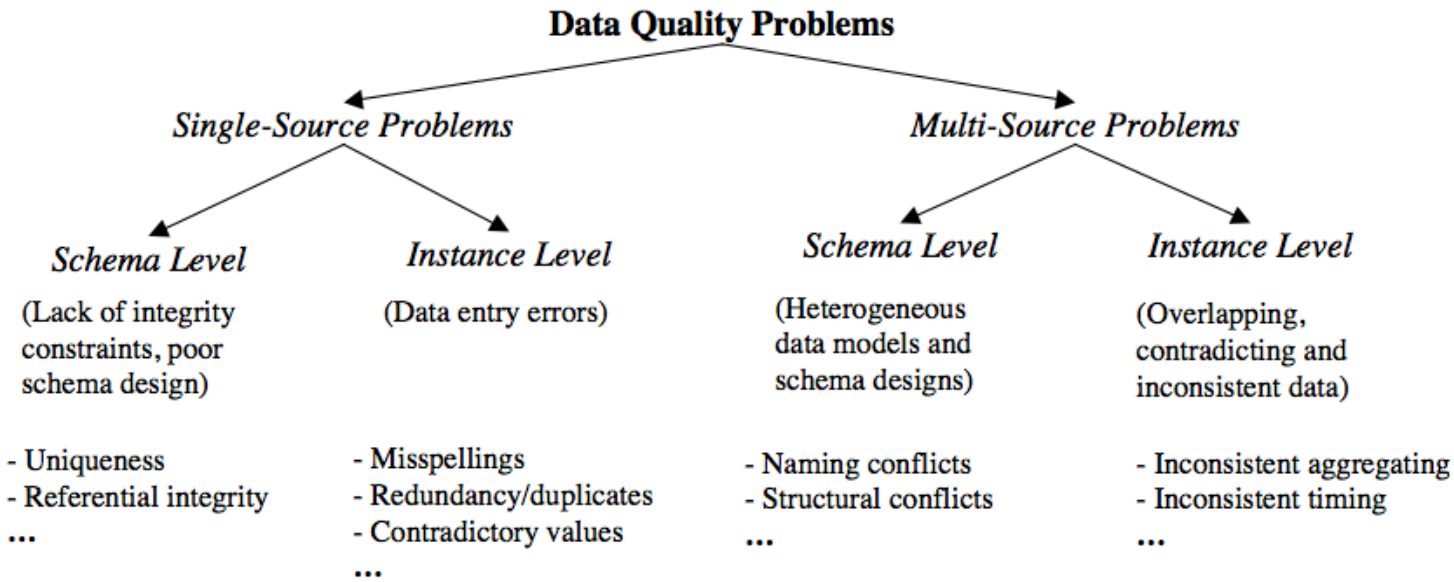

A typical Trajectory
Wrangling
- Cleanse
- Merge
- Adapt
- Evaluate Usability of data
Analysis
- Visualize
- Analyze
Ideal Tool combining wrangling and analysis
Visual (or not) Wrangling Tools
Visual Analytics Tools
Original Data Sets
Analytics Processes
1. Diagnose
2. Transform
3. 1st load in analysis application
4. Start of analysis
5. Repeat with new data
6. Document
Updated Data Sets
raw data
usable data
Trail of data transformations
usable data + findings
Trail of analysis = Insight Provenance

Data Quality

- Different sources are developed separately, and maintained by different people
- Issue 1: Mapping information across sources (schema mapping/transformation)
  - Naming conflicts: same name used for different objects
  - Structural conflicts: different representations across sources
  - We will cover this later
- Issue 2: Entity Resolution: Matching entities across sources
- Issue 3: Data quality issues
  - Contradicting information, Mismatched information, etc.

From Data Cleaning: Problems and Current Approaches

Data Quality Problems



Single-source problems

- Depends largely on the source
- Databases can enforce constraints, whereas data extracted from files or spreadsheets, or scraped from webpages is much more messy
- Types of problems:
  - Ill-formatted data, especially from webpages or files or spreadsheets
  - Missing or illegal values, Misspellings, Use of wrong fields, Extraction issues (not easy to separate out different fields)
  - Duplicated records, Contradicting Information, Referential Integrity Violations
  - Unclear default values (e.g., data entry software needs something)
  - Evolving schemas or classification schemes (for categorical attributes)
  - Outliers

Examples

Scope/Problem		Dirty Data	Reasons/Remarks
Attribute	Missing values	phone=9999-999999	unavailable values during data entry (dummy values or null)
	Misspellings	city="Liipzig"	usually typos, phonetic errors
	Cryptic values, Abbreviations	experience="B"; occupation="DB Prog."	
	Embedded values	name="J. Smith 12.02.70 New York"	multiple values entered in one attribute (e.g. in a free-form field)
	Misfielded values	city="Germany"	
Record	Violated attribute dependencies	city="Redmond", zip=77777	city and zip code should correspond
Record type	Word transpositions	$name_1$= "J. Smith", $name_2$="Miller P."	usually in a free-form field
	Duplicated records	emp_1=(name="John Smith",...); emp_2=(name="J. Smith",...)	same employee represented twice due to some data entry errors
	Contradicting records	emp_1=(name="John Smith", bdate=12.02.70); emp_2=(name="John Smith", bdate=12.12.70)	the same real world entity is described by different values
Source	Wrong references	emp=(name="John Smith", deptno=17)	referenced department (17) is defined but wrong

Multi-source problems

- Different sources are developed separately, and maintained by different people
- Issue 1: Mapping information across sources (schema mapping/transformation)
  - Naming conflicts: same name used for different objects
  - Structural conflicts: different representations across sources
  - We will cover this later
- Issue 2: Entity Resolution: Matching entities across sources
- Issue 3: Data quality issues
  - Contradicting information, Mismatched information, etc.

Missing and Repeated Values

- Missing values and repeated values are common
- Many ML algorithms don’t deal with missing values
- Repeated values bias results

Example: Missing Values

Col1	Col2	Col3	Col4	Col5
12456	0.99	Male	43	Small
98567	1.23		55	Medium
34567	9999	Female	NA	Large
67231	0.72	Male	35	?

Treating Missing Values

- Remove rows
- Substitute a specific value
- Interpolate values
- Forward fill
- Backward fill
- Impute

Missing and Repeated Values

- Clean Missing Data module
- With R – is.na()
- With Python – pandas.DataFrame.isnull()

Example: Repeated Values

Key Col	Col2	Col3	Col4	Col5
12456	0.99	Male	43	Small
98567	1.23	Male	55	Medium
34567	1.55	Female	43	Large
34567	1.55	Female	43	Large
34567	1.55	Female	43	Large
34567	.78	Male	43	Large
67231	0.72	Male	35	Small

Repeated Values

- Clean Repeated Values module
- With R – data.frame[!duplicated(),]
- With Python – DataFrame.drop_duplicates()

Univariate Outliers

- Using Quantifiable Measurements
  - Center (e.g., mean), Dispersion (e.g., standard deviation), Skew
  - **Statistics** to minimize the effect of corrupted data
  - Median distance of all data from the median
- A Reasonable Approaches
  - Visual Examination (e.g., plot a histogram)
  - 1.4826x MAD (Median distance from the Median)
  - Visual Validation

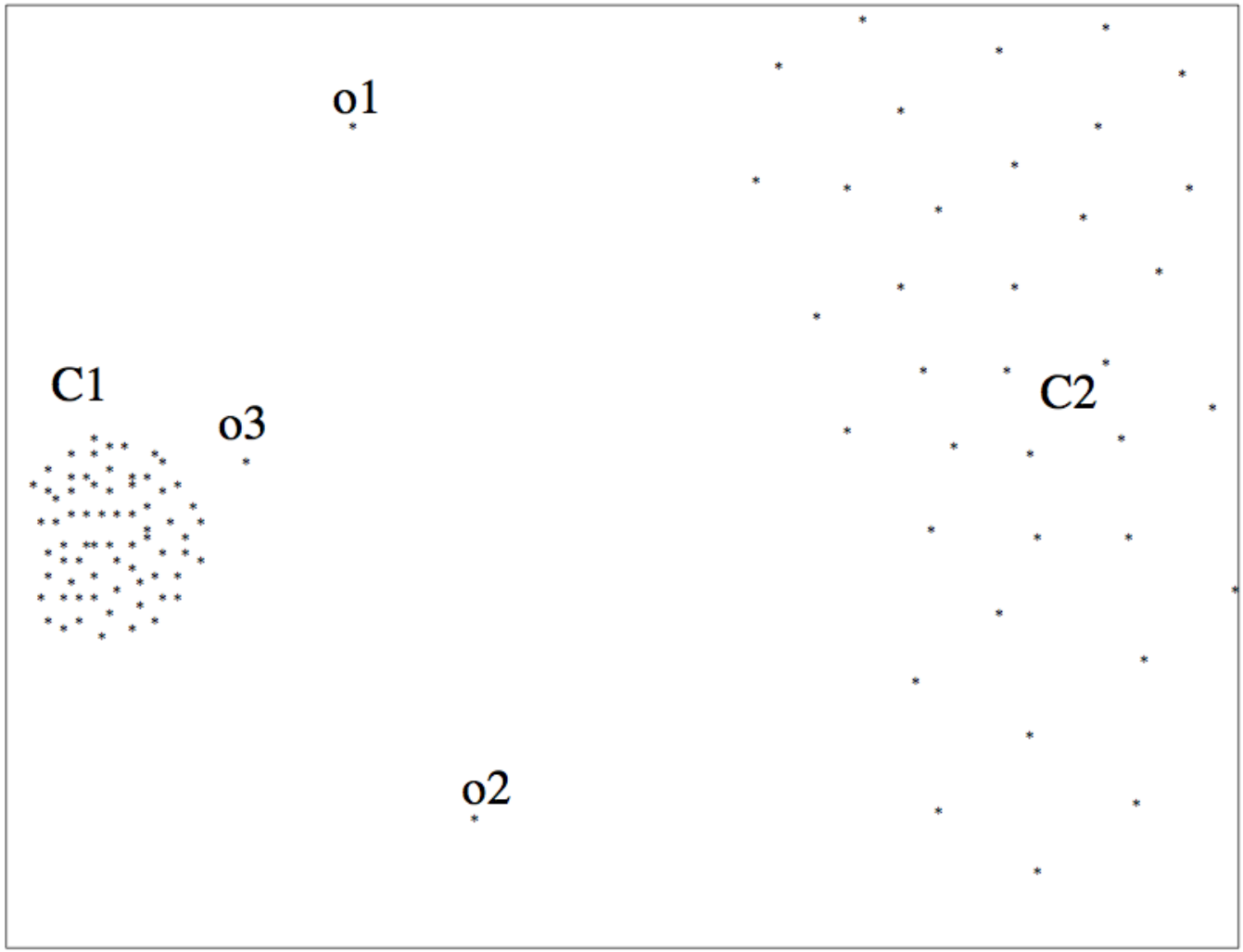
Univariate Outliers

- If data appears to be not normally distributed:
  - Distance-based methods
    - Look for data points that do not have many neighbors
  - Density-based methods:
    - Define *density* to be average distance to *k* nearest neighbors
    - *Relative density* = node/average density of its neighbors
    - Use relative density to decide if a node is an outlier
- Caveat
  - Most of these techniques start breaking down as the dimensionality of the data increases
  - *Curse of dimensionality*
  - Can project data into lower-dimensional space and look for outliers there
    - Not as straightforward

Multivariate Outliers

- Multi-variate normal distribution
  - Analogous to univariate
  - Defined by a mean
  - Measures how far the point x is from the multivariate normal distribution
  - Look for points that are too far away
- Mean/Covariance are not Robust
  - Iterative approach: remove points with high distance, recompute the mean and covariance
  - Several other general approaches
  - No clear guidelines here -- need to try different techniques based on the data

Outliers



Other Outliers

- Timeseries Outliers
  - Often the data is in the form of a timeseries
  - Can use the historical values/patterns in the data to flag outliers
  - Rich literature on *forecasting* in timeseries data
- Frequency-based Outliers
  - An item is considered a "heavy hitter" if it is much more frequent than other items
  - In relational tables, can be found using a simple *group by-count*
  - Often the volume of data may be too much
    - Approximation techniques often used
    - To be discussed sometime later in the class
- Outlier detection continues to be a major research area

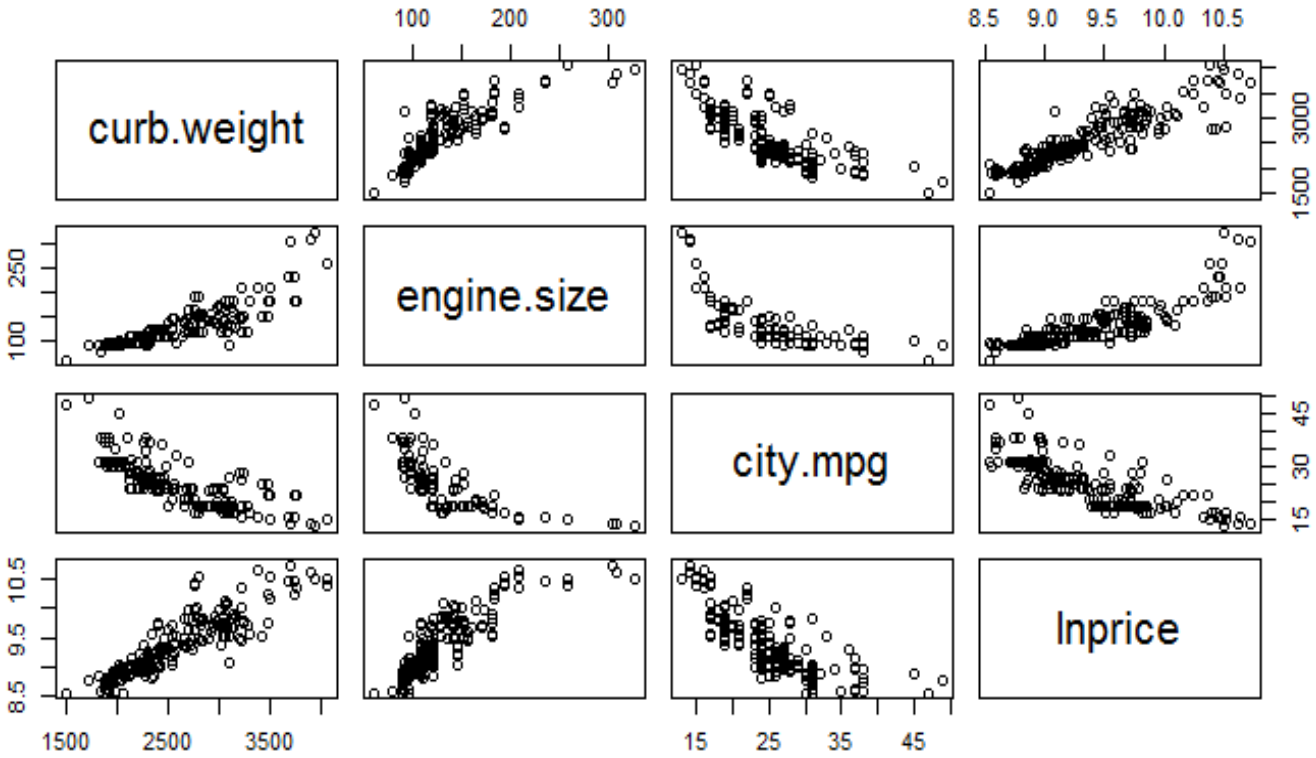
Outliers and Errors

- Errors and outliers can bias model training
- Many possible sources of errors
  - Erroneous measurements
  - Entry errors
  - Transposed values in table
- Discover and evaluate with summary statistics and visualization

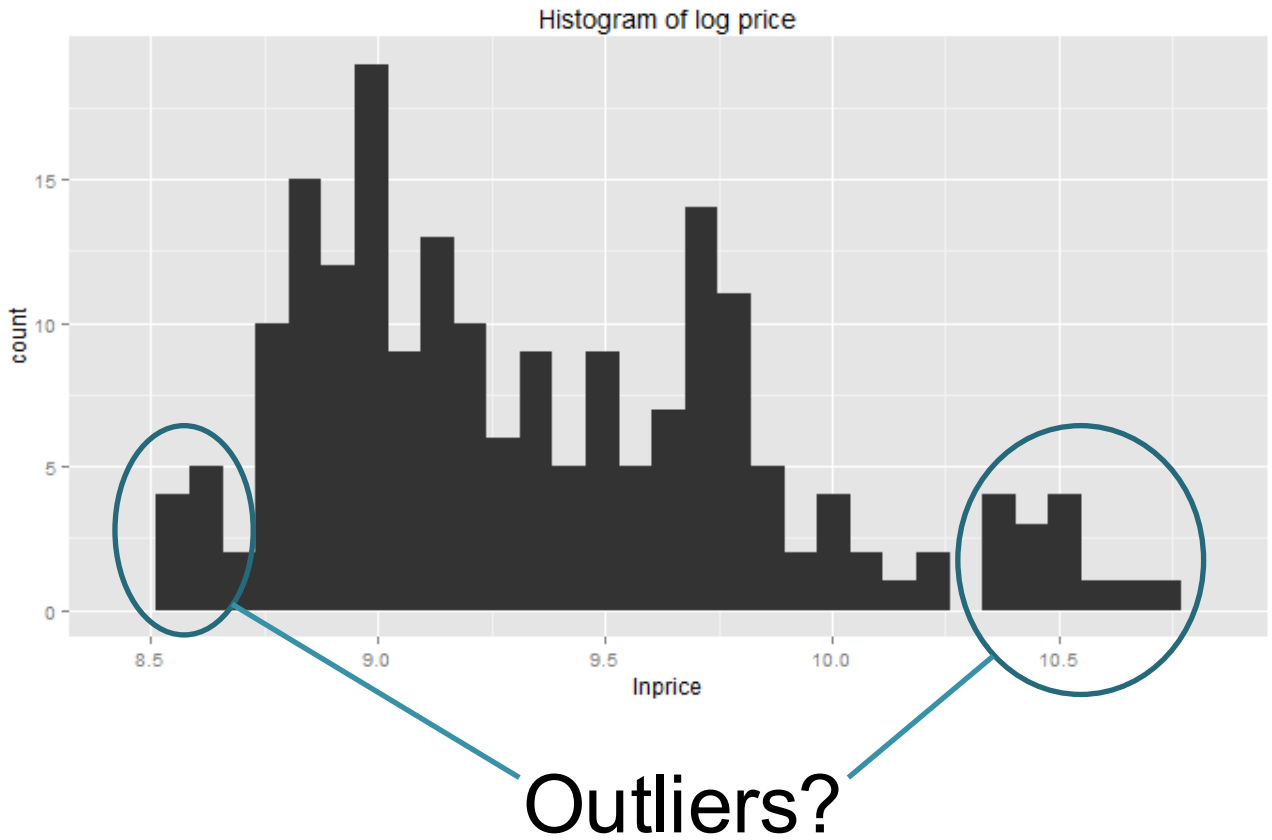
Visualizing Outliers

- Scatter plot matrix helps validate outliers
- R – pairs plot
- Python – pandas.tools.plotting.scatter_matrix

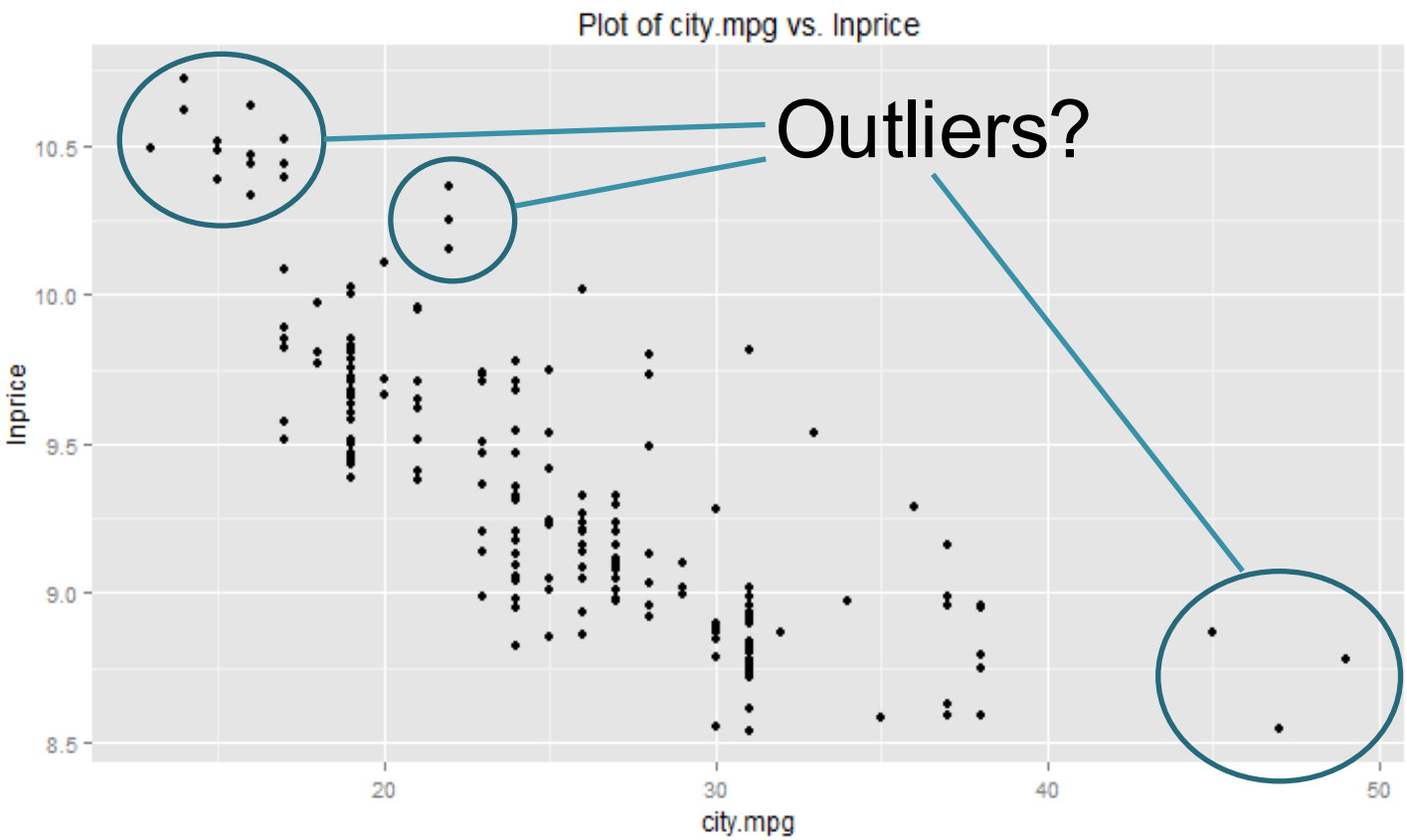
Visualizing Outliers



Identify Outliers and Errors



Identify Outliers and Errors



Clean Outliers and Errors

- Error treatments
  - Censor
  - Trim
  - Interpolate
  - Substitute
- Clip Values module
- With R
- With Python

Removing Outliers

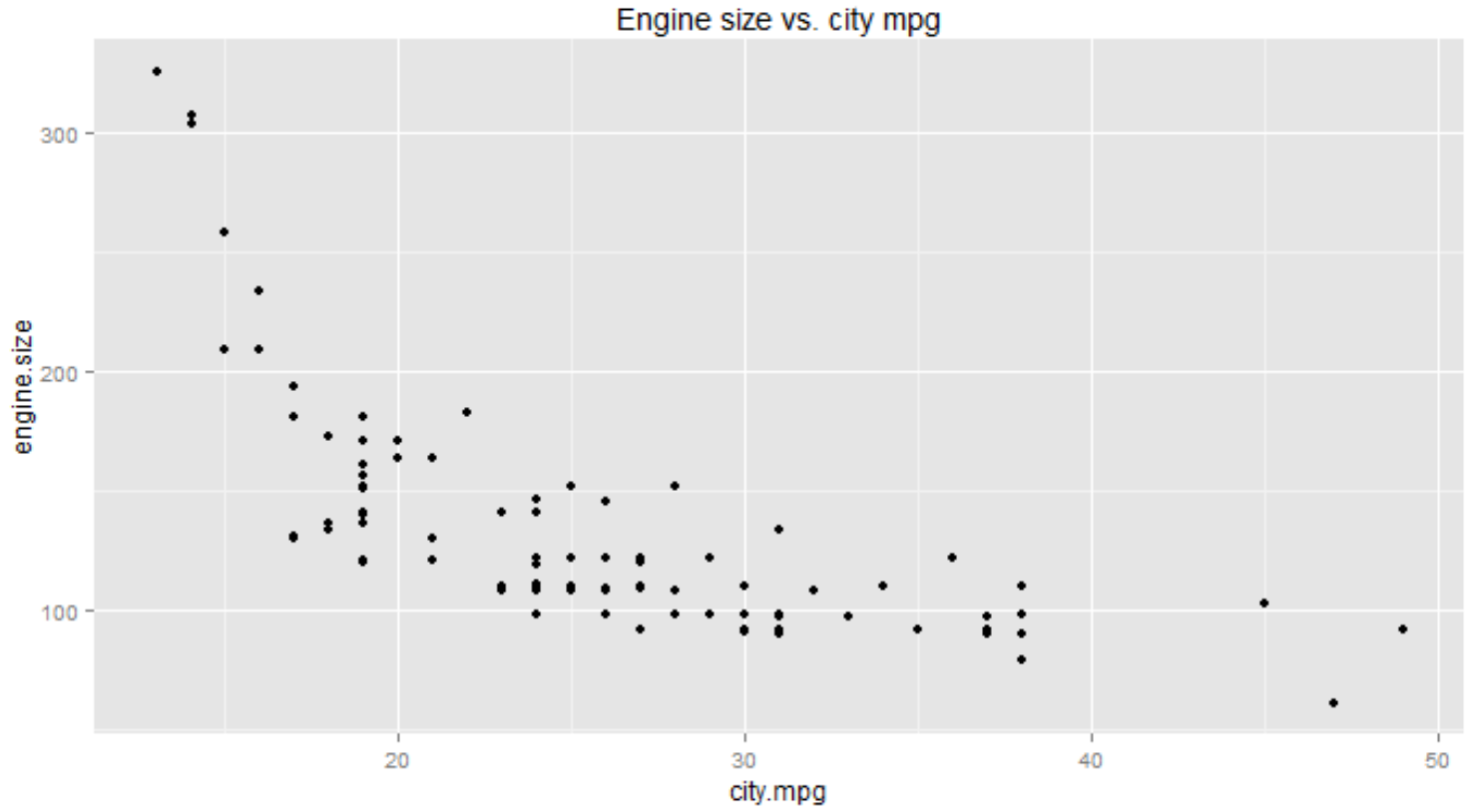
R: data.frame = data.frame[filter.expression,]

```
library(dplyr)
frame1 <- frame1 %>% filter(Col1 > 40) %>%
                     filter(Col2 < 30) %>%
                     filter(Col3 < 3)
```

Python: DataFrame = DataFrame[filter_expression]

```
frame1 = frame1[(frame1["Col1"] > 40.0) &
                (frame1["Col2"] < 30.0) &
                (frame1["Col3"] < 3.0)]
```

Scatter plot of two columns



Entity Resolution

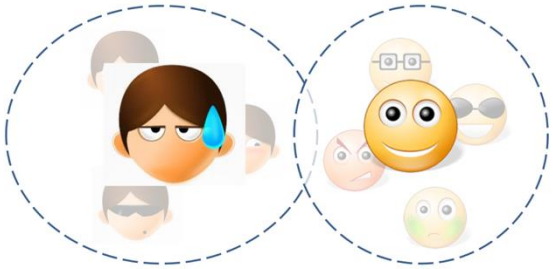
- Different manifestations of the same real-world object
  - Also called: identity reconciliation, record linkage, deduplication, fuzzy matching, Object consolidation, Coreference resolution, and several others
- Motivating examples
  - Postal addresses
  - Entity recognition in NLP/Information Extraction
  - Identifying companies in financial records
  - Comparison shopping
  - Author disambiguation in citation data
  - Connecting up accounts on online networks
  - Crime/Fraud Detection
  - Census
  - ...

Entity Resolution

- Entity Resolution
  - Often, actions taken based on extracted data
  - Cleaning up data by entity resolution can show structure that may not be apparent before
- Challenges
  - Data is naturally ambiguous (e.g., names, postal addresses)
  - Abbreviations/data truncation
  - Data entry errors, Missing values, Data formatting issues complicate the problem
  - Heterogeneous data from many diverse sources
- No magic bullet
  - Approaches fairly domain-specific
  - Be prepared to do a fair amount of manual work

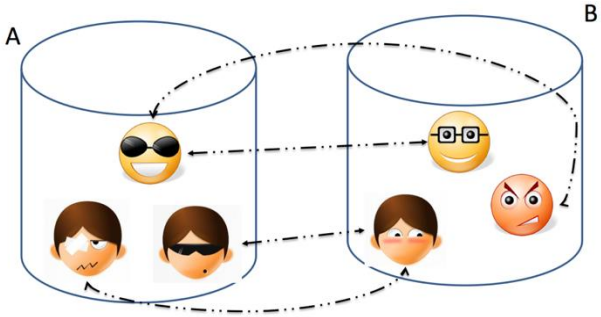
Entity Resolution Example

- Setup
  - Real world: there are entities (people, addresses, businesses)
  - We have a large collection of noisy, ambiguous "references" to those entities (also called "mentions")
  - Somewhat different techniques, but a lot of similarities
- Deduplication
  - Cluster records/mentions that correspond to the same entity
  - Choose/construct a cluster representative
    - Averaging may work for numerical attributes, but what about string attributes?



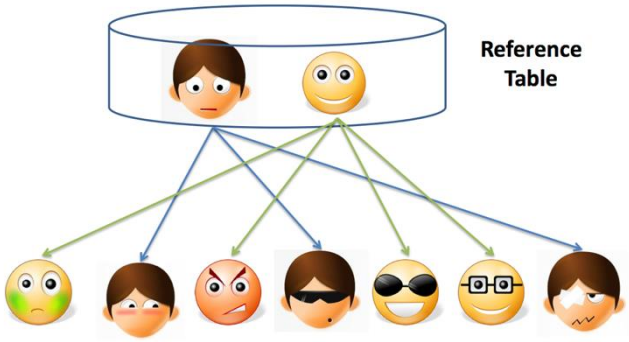
Entity Resolution Example

- Setup
  - Real world: there are entities (people, addresses, businesses)
  - We have a large collection of noisy, ambiguous "references" to those entities (also called "mentions")
  - Somewhat different techniques, but a lot of similarities
- Record Linkage
  - Match records across two different databases (e.g., two social networks, or financial records w/ campaign donations)
  - Typically assume that the two databases are fairly clean



Entity Resolution

- Setup
  - Real world: there are entities (people, addresses, businesses)
  - We have a large collection of noisy, ambiguous "references" to those entities (also called "mentions")
  - Somewhat different techniques, but a lot of similarities
- Reference Matching
  - Match "references" to clean records in a reference table
  - Commonly comes up in "entity recognition" (e.g., matching newspaper article mentions to names of people)



Entity Resolution: Data Matching

- One of the key issues is finding similarities between two references
  - What similarity function to use?
- Distance Functions
  - Min number of changes to go from one reference to another
    - A change is defined to be: a single character insertion or deletion or substitution
    - May add transposition
  - Many adjustments (e.g., higher weights to changes at the start)
  - Not cheap to compute, especially for millions of pairs
- Set Similarity
  - Some function of intersection size and union size
  - E.g., Jaccard distance = size of intersection/size of union
  - Much faster to compute
- Vector Similarity
  - Cosine similarity – we'll talk about this much more in NLP lectures

Entity Resolution: Data Matching

- q-Grams
  - Find all length-q substrings in each string
  - Use set/vector similarity on the resulting set
- Soundex: Phonetic Similarity Metric
  - Homophones should be encoded to the same representation so spelling errors can be handled
  - Robert and Rupert get assigned the same code (R163), but Rubin yields R150
- May need to use Translation Tables
  - To handle abbreviations, nicknames, other synonyms
- Often Requires Domain-specific Functions
  - E.g., geographical locations, postal addresses
  - Much work on computing distances between XML documents etc.

Entity Resolution: Algorithms

- Simple threshold method
  - If the distance below some number, the two references are assumed to be equal
  - May review borderline matches manually
- Can be generalized to rule-based

$$(s(\text{GivenName})[r_i, r_j] \geq 0.9) \wedge (s(\text{Surname})[r_i, r_j] = 1.0) \wedge (s(\text{BMonth})[r_i, r_j] = 1.0) \wedge (s(\text{BYear})[r_i, r_j] = 1.0) \Rightarrow [r_i, r_j] \rightarrow \text{Match}$$

$$(s(\text{GivenName})[r_i, r_j] \geq 0.7) \wedge (s(\text{Surname})[r_i, r_j] \geq 0.8) \wedge (s(\text{BDay})[r_i, r_j] = 1.0) \wedge s(\text{BMonth})[r_i, r_j] = 1.0) \wedge (s(\text{BYear})[r_i, r_j] = 1.0) \Rightarrow [r_i, r_j] \rightarrow \text{Match}$$

$$(s(\text{GivenName})[r_i, r_j] \geq 0.7) \wedge (s(\text{Surname})[r_i, r_j] \geq 0.8) \wedge (s(\text{StrName})[r_i, r_j] \geq 0.8) \wedge (s(\text{Suburb})[r_i, r_j] \geq 0.8) \Rightarrow [r_i, r_j] \rightarrow \text{Match}$$

$$(s(\text{GivenName})[r_i, r_j] \geq 0.7) \wedge (s(\text{Surname})[r_i, r_j] \geq 0.8) \wedge (s(\text{BDay})[r_i, r_j] \leq 0.5) \wedge (s(\text{BMonth})[r_i, r_j] \leq 0.5) \wedge (s(\text{BYear})[r_i, r_j] \leq 0.5) \Rightarrow [r_i, r_j] \rightarrow \text{Non-Match}$$

$$(s(\text{GivenName})[r_i, r_j] \geq 0.7) \wedge (s(\text{Surname})[r_i, r_j] \geq 0.8) \wedge (s(\text{StrName})[r_i, r_j] \leq 0.6) \wedge (s(\text{Suburb})[r_i, r_j] \leq 0.6) \Rightarrow [r_i, r_j] \rightarrow \text{Non-Match}$$

Entity Resolution: Algorithms

- May want to give more weight to rarer words
  - More naturally applicable to record linkage problem
  - If two records match on a rare name, they are likely a match
  - Can formalize this as "probabilistic record linkage"
- Constraints
  - May need to be satisfied, but can also be used to find matches
  - Often have constraints on the matching possibilities
  - Transitivity: M1 and M2 match, and M2 and M3 match, and M1 and M3 must match
  - Exclusivity: M1 and M2 match --> M3 cannot match with M2
  - Other types of constraints:
    - E.g., if two papers match, their venues must match

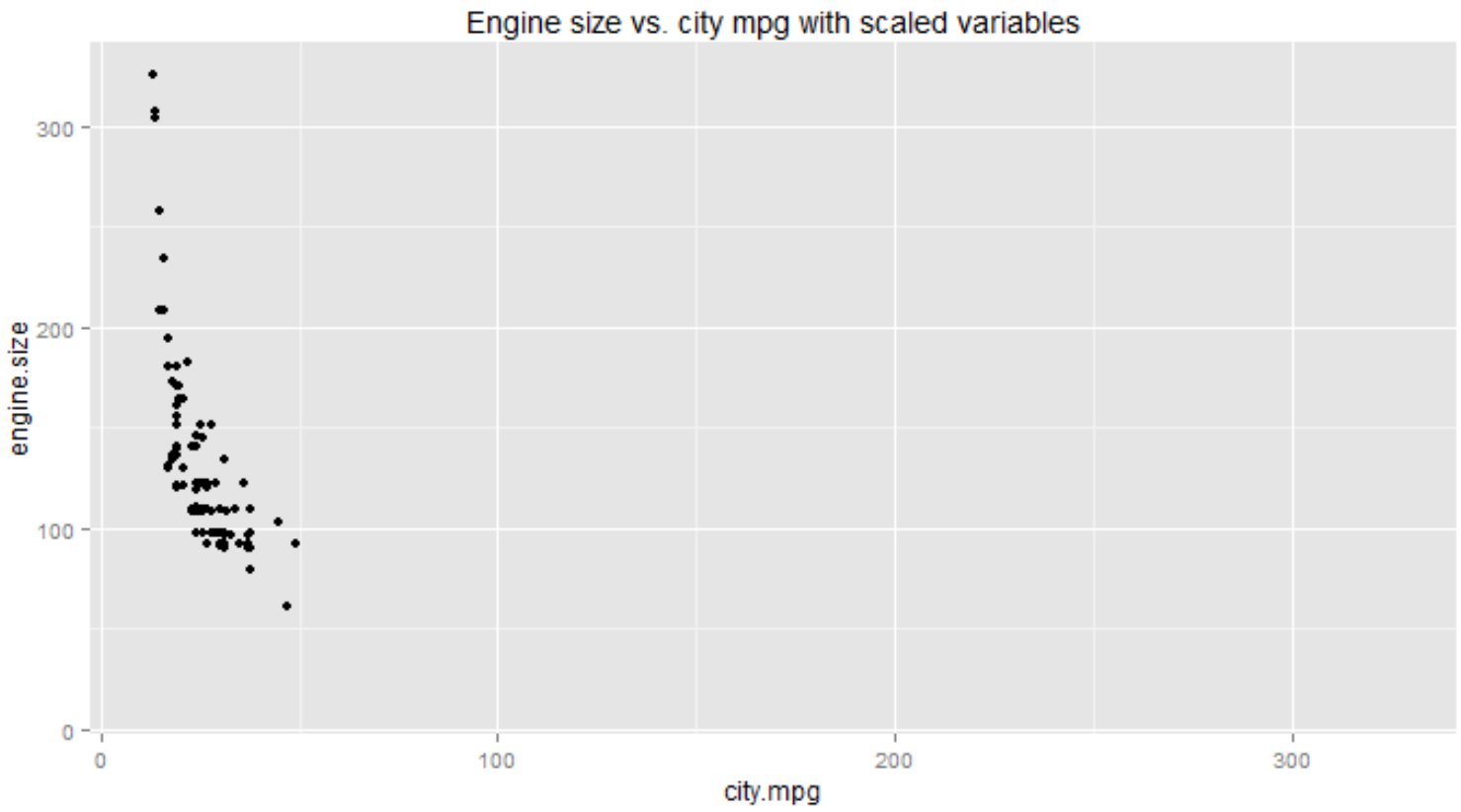
Entity Resolution: Algorithms

- Clustering-based Techniques:
  - Deduplication is basically a clustering problem
  - Can use clustering algorithms for this purpose
  - But most clusters are very small
- Crowdsourcing
  - Humans are often better at this task
  - Can use one of the crowdsourcing mechanisms (e.g., Mechanical Turk) for getting human input on the difficult pairs
  - Quite heavily used commercially (e.g., to disambiguate products, restaurants, etc.)

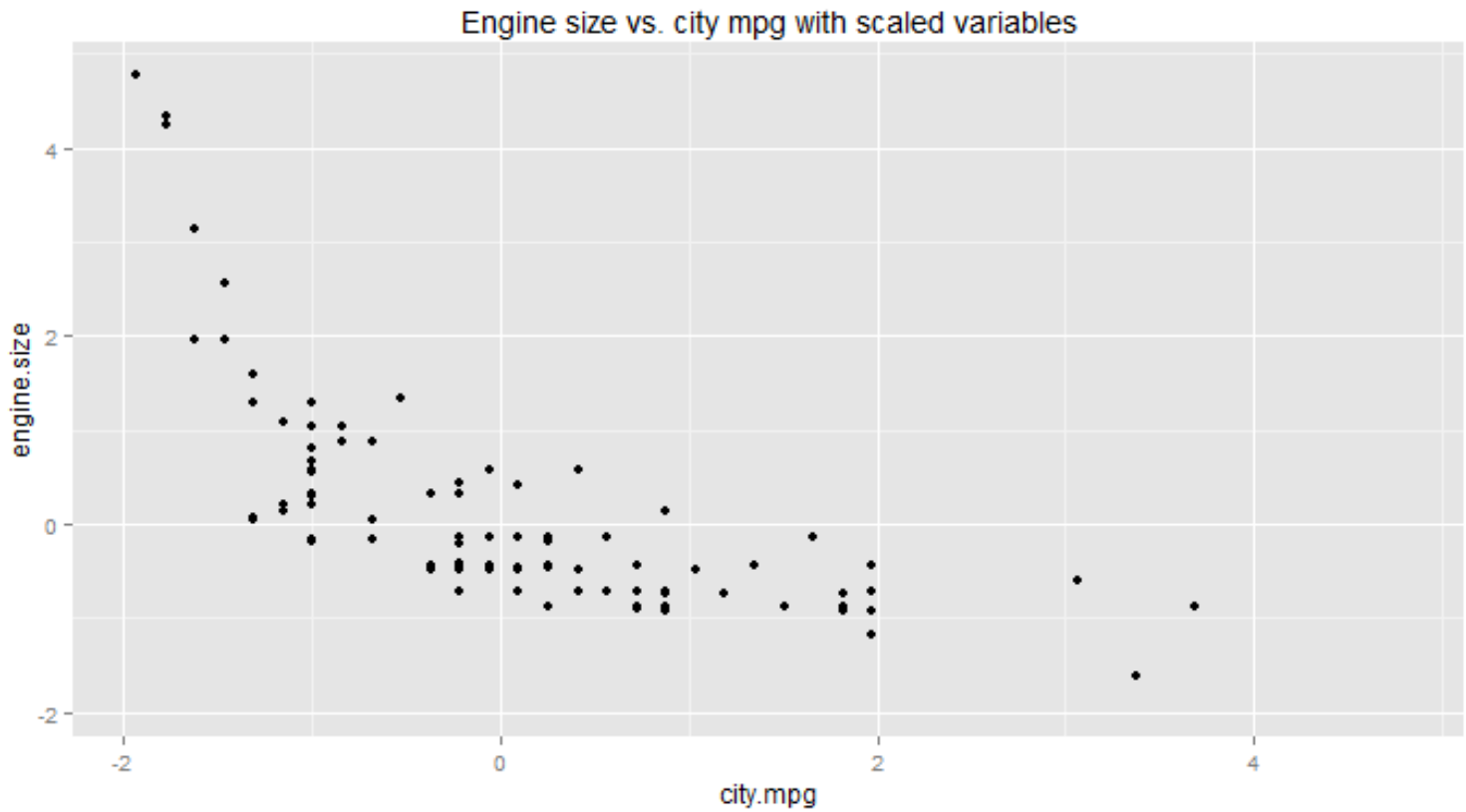
Scaling

- Easy-to-Evaluate Criterion
  - Restrict the pairs considered further
  - May lead to false negative (i.e., missed matches) depending on how noisy the data is
- Domain-specific knowledge likely to be more useful in practice

Unscaled data biases model



Scaled data biases model



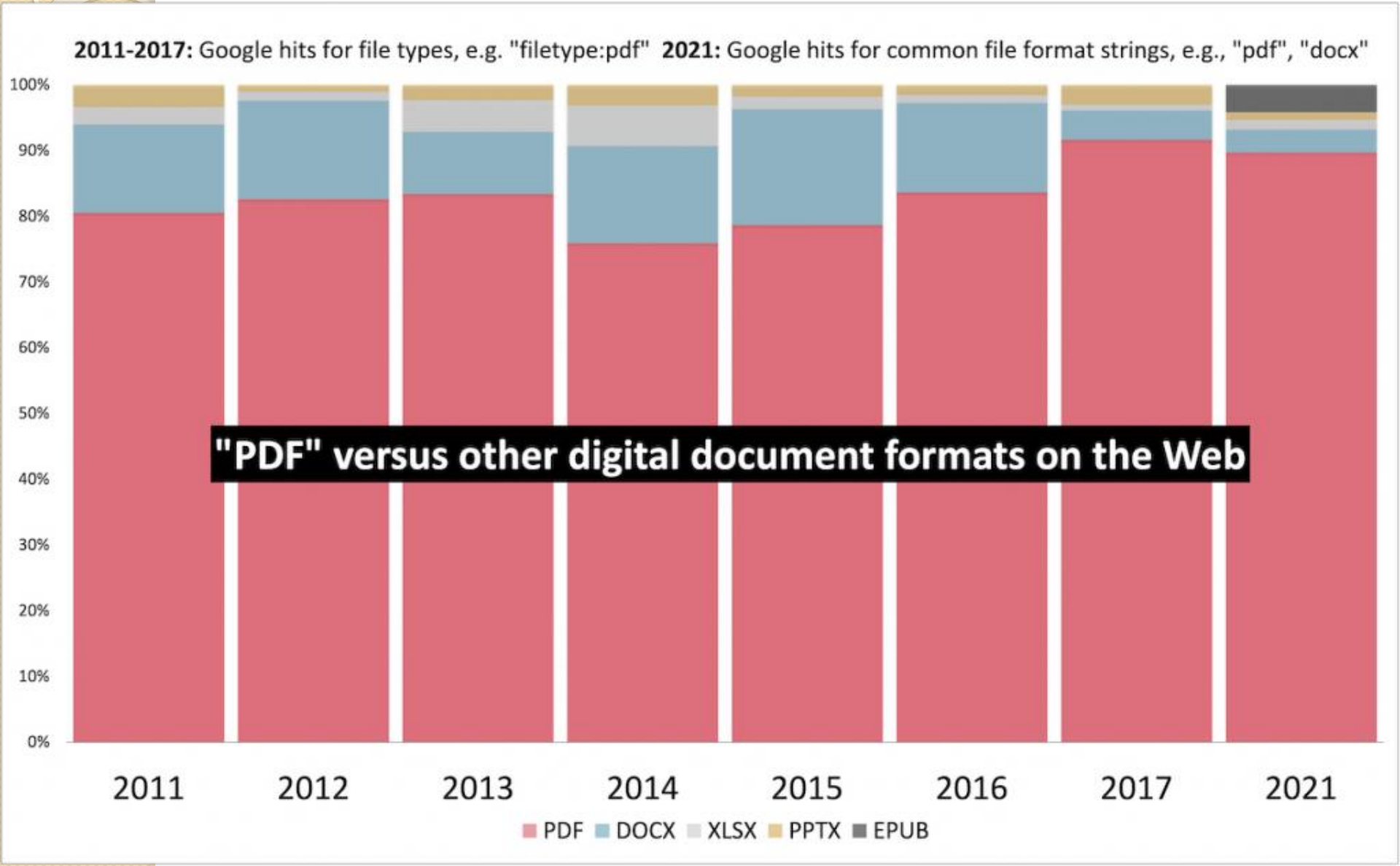
Scaling

- Normalize Data module
- With R:
  - `scale()`
- With Python:
  - `scikit-learn.preprocessing.Scale()`

Data Preparation

- Get data into a structured form suitable for analysis
  - Variously called: data preparation, data munging, data curation
  - Also often called ETL (Extract-Transform-Load) process
- Often the step where majority of time (80-90%) is spent
- Key steps:
  - Scraping: extracting information from sources, e.g., webpages, spreadsheets
  - Data transformation: to get it into the right structure
  - Data integration: combine information from multiple sources
  - Information extraction: extracting structured information from unstructured/text sources
  - Data cleaning: remove inconsistencies/errors

Example: Common Internet Documents

- Non-ASCII Doc
  - 90% PDF
  - 3% DOCX
  - 2% XLSX
  - 1% PPTX
  - 4% EPUB
- ASCII Text
  - HTML
  - XML
  - JSON
  - CSV
  - Unstructured

Digital Documents on the Web